CSCI 350:
Fundamentals of Operating Systems

Course Introduction, Process Abstraction

Prof. Tatyana Ryutov

This content is protected and may not be shared, uploaded, or distributed.

Lecture Outline

- Course information
- What is OS?
- Address space abstraction
- Process abstraction
- Syscall intro

© Tatyana Ryutov

Synchronous Session Recording

- Live class sessions will be recorded and made available to students through Brightspace/D2L

- Zoom link: https://usc.zoom.us/j/93731129104?pwd=Ez6nbrzPSWk7KrWliSx8VAM0Frn3PF.1
- Cloud recordings should be found under Brightspace/Course ToolsUSC zoom

- Please remember that USC policy prohibits sharing of any synchronous and asynchronous course content outside of the learning environment
- As a student, you are responsible for the appropriate use and handling of these recordings under existing SCampus policies regarding class notes (https://policy.usc.edu/scampus-part-c/)
- These rules will be strictly enforced, and violations will be met with the appropriate disciplinary sanction

USC Viterbi School of Engineering

© Tatyana Ryutov

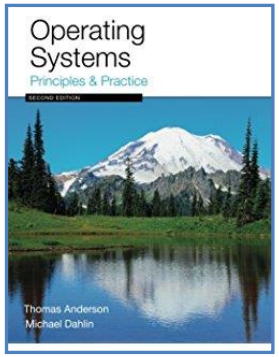
Course Team

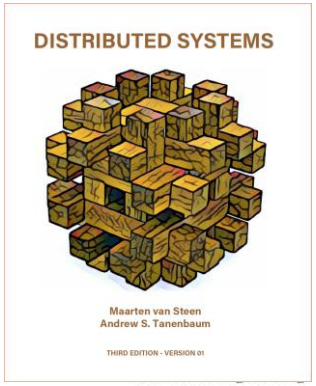
- Instructor: Prof. Tatyana Ryutov, **tryutov@usc.edu**
  - Office hours: Thursday 3-5pm
  - https://usc.zoom.us/j/98603673241?pwd=bvaXO61r7YlbQCjTtsX7zeB8cXZVlK.1
  - In-person by request
- TAs:
  - Tianwen tianwenf@usc.edu
  - Tuan tkngo@usc.edu
- CPs (see Piazza)
  - Your best source for project related questions

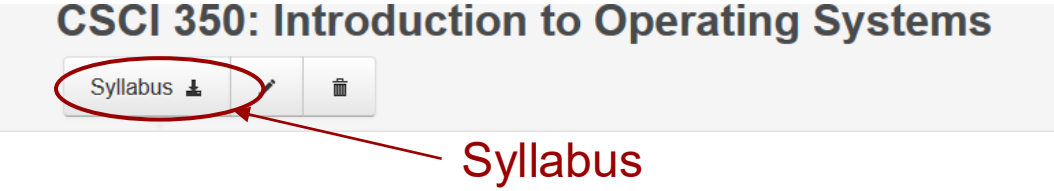
© Tatyana Ryutov

What should you have?

- Required textbooks:
  - **T1**: “Operating Systems: Principles and Practice” by Thomas Anderson and Mike Dahlin, 2nd edition
  - **T2**: “Operating System Concepts” by Abraham Silberschatz, Peter Galvin, and Greg Gagne, 10th edition
  - **T3**: “Distributed Systems” by M. van Steen and A.S. Tanenbaum, 3rd edition
- Access to Brightspace
  - Grades, exams, lecture recordings will be posted there
- Access to Piazza
  - piazza.com/usc/fall2026/csci350
  - Course materials will be posted there
  - A great place to ask questions and get answers
- A GitHub account
  - Talk to Logan







© Tatyana Ryutov

Syllabus: Course Scheduler

Week	Topic	Readings
1/16 Lec1	Introduction Process abstraction System calls intro	Introduction Process abstraction System calls intro
1/23 Lec2	Dual mode operation Context switch System call implementation	T1: Chapters 2.2; 2.3; T1: Chapters 2.4.1-2.4.5 T1: Chapter 2.6
1/30 Lec3	Kernel organization Virtual machines Process management Interrupts	T2: Chapters 2.8.1-2.8.3 T2: Chapter 18 T1: Chapters 3.1-3.3 T2: Chapter 1.2.1
2/6 Lec4	Concurrency and threads	T1: Chapters 4.1-4.8
2/13 Lec5	Synchronization	T1: Chapters 5.1-5.5; 5.7; 5.8
2/20	**Midterm1**	
2/27 Lec6	CPU scheduling	T1: Chapters 7.1
3/6 Lec7	Advanced Synchronization	T1: Chapter 6.5 T2: Chapter 8
3/13 Lec8	Memory Management	T1: Chapters 8.1-8.3
3/20	Spring recess, no class	
3/27 Lec9	Storage Devices File System implementation	T1: Chapter 12 T1: Chapters 11; 13.1- 13.3.3
4/3 Lec10	COW File Systems Reliable Storage Distributed Systems intro	T1: Chapter 13.3.4 T1: Chapter 14 T2: Chapters 19.1-19.3
4/10 Lec11	Remote Procedure Call Distributed Mutual exclusion Distributed parallel computing Map-reduce computing paradigm	T3: 4.2 T3: 6.3 MapReduce paper
4/17	**Midterm2**	
4/24 Lec12	Protection and Security	T2: Chapters 14.4-14.6; 15.1-15.3; 15.6-15.8
5/1 Lec13	Trusted OS design	
5/6	**Final Examination 2-4pm**	

T1: "Operating Systems: Principles and Practice"
T2: "Operating System Concepts"
T3: "Distributed Systems"

USC Viterbi School of Engineering

© Tatyana Ryutov

Guidelines for Students

- Student deliverables
  - Two midterms and final exam
    - Each exam is 2 hours
    - Final exam is cumulative, midterms are not
  - Five programming projects (C language)
    - Two are optional
    - For each project will you have to answer Free Response Questions
      - You will submit your answers to Brightspace individually
- Required readings
  - Primarily from the three textbooks
  - You will not be tested on the material in the textbooks that we did not cover in class
  - Recommended (optional) material will be posted

Class will proceed at a brisk pace and often be more focused on providing extra intuition and discussions rather than rehashing the textbook content in great detail
Read before class!

USC Viterbi School of Engineering

© Tatyana Ryutov

Grading Schema

Midterm 1: 20%
Midterm 2: 20%
Final exam: 25%
Class participation: 5%
Projects: 30% + 10 % extra credit

- Project 1: 5%
- Projects 2: 15%
- Project 3: 5% (extra credit)
- Projects 4: 5% (extra credit)
- Project 5: 10%

You van get partial credit for extra credit projects

Total **100% + 10% extra credit**

USC Viterbi School of Engineering

© Tatyana Ryutov

Letter Grade Assignment

- No curve!
- This grade assignment will not change
- We round the scores up, e.g., if your total score is 82.01, you will get A-

92 - 100	= A	59 - 55	= C
91 - 83	= A -	54 - 50	= C-
82 - 77	= B+	49 - 45	= D+
76 - 71	= B	44 - 40	= D
70 - 65	= B-	39 - 35	= D-
64 - 60	= C+	Below 35 is an F	

USC Viterbi School of Engineering

© Tatyana Ryutov

Important Dates

- February 6: Project1 due
- February 20: Midterm1,12-2pm (online)
- March 13: Project2 due
- April 3: Project3 due
- April 17: Midterm2,12-2pm (online)
- April 17: Project4 due
- May 1: Project5 due
- May 6: Final Exam, 2-4pm (online)

© Tatyana Ryutov

Exam Policies

- All exams will be administered remotely via Brightspace
  - Posted in Activities/Quizzes
- The exams are held on the days listed, the dates are NOT movable
  - Students requiring alternate exam arrangements must make such requests within the first two weeks of the term, or as soon as possible after knowing of the conflict or requirement
    - This requires appropriate documentation
  - Exams will not be excused for reasons considered to be within your control
- Each exam is individual effort, closed-book, closed notes
  - Cheat sheet
    - For each exam you can have one (both sides) handwritten cheat sheet that you can keep
- **No AI tools allowed in this class!**
- **No AI tools can be used to create your cheat sheet**
- Practice midterm1 and 2 exams will be posted; no practice final exam
- Regrades:
  - For non-final exams and projects 1-4 you will have 7 days since the date exam or project grades are released to request a regrade
  - For final exam and project 5 you will have 1 day to request a regrade

USC Viterbi School of Engineering

© Tatyana Ryutov

Remote Exam Policies

- Access the exam via the Brightspace available in Activities/Quizzes on the specified day and time.
- Exam is closed book/notes.
- You'll have 2hrs + 15-min grace period to complete the exam.
- After 135 minutes, your exam will be automatically submitted. You will not be able to pause the exam.
- One restroom break is allowed (5 minutes max).
- You can wear headphones.
- You can have 1 sheet of scratch paper
- You are allowed one 8.5" x 11" handwritten note sheet (both sides). NO TYPED NOTES ARE ALLOWED!
- You may post your exam-related questions during the exam via Piazza. Use "private question" setting.
- During the exam you'll record yourself: **share your screen** where you'll be inputting the answers, microphone and camera must be on. We should be able to see your face.
- If you experience a problem with the recording, let us know immediately via Piazza!
- You can copy your answers to a text editor and save them as a precaution.
- **Only the Brightspace, Piazza, and text editor should be opened. All other browser tabs must be closed.**
- **No second screens are allowed.**
- After the exam is over and your recording is ready, submit the link via a Google form on Piazza.
- You need to submit a scan of both sides of your cheat sheet along with your recording
- **We will not grade your exam if you do not provide your recording, or if the recording does not satisfy the above requirements.**
- **If you have never recorded your Zoom meeting, practice well before the exam!**
- To schedule the recording login into Zoom using your USC credentials, record to the cloud.
- In order to not forget to record yourself, select "record the meeting automatically" when scheduling your meeting.

USC Viterbi School of Engineering

© Tatyana Ryutov

USC Final Examination policy

- No student is permitted to omit or take early a final examination and no instructor is authorized to permit a student to do so.
- Students should plan in advance to avoid scheduling conflicts in their final examinations. If a student is scheduled for two final examinations at the same time, the student should request to take one of the examinations on a different day or time. If a student is scheduled for more than two final examinations in one day, the student may request to take one of the exams on a different day or time. In either situation the student must contact the professors involved no later than two weeks prior to the scheduled examination date and request an accommodation. If an accommodation cannot be arranged, the student should contact the office of Academic Programs at academic.programs@usc.edu.

USC Viterbi School of Engineering

© Tatyana Ryutov

Submitting Assignments

- Submit via our GitHub Classroom
- No other submission methods will be accepted
  - The last commit before deadline will be the default submission for grading
- To gain access to the organization:
  - Create a GitHub account
  - Read the project rules, posted on Piazza
  - Take the rules quiz
  - We will create your account

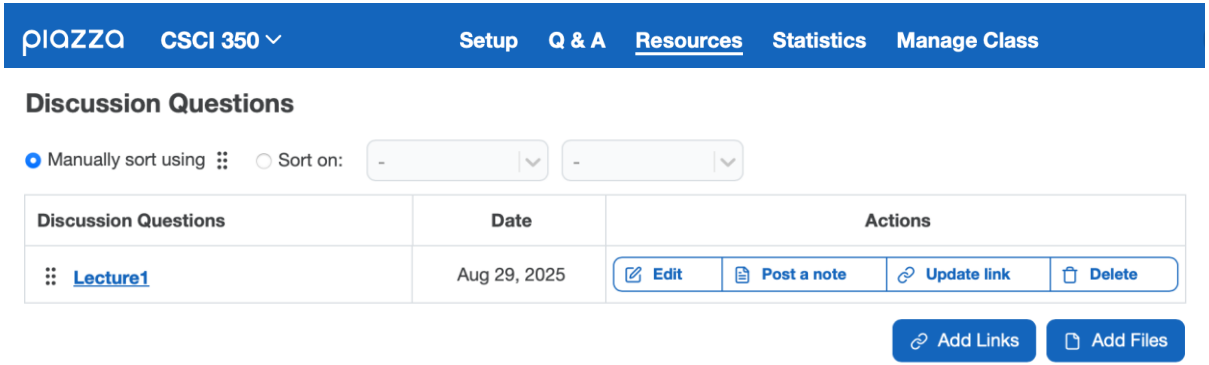
© Tatyana Ryutov

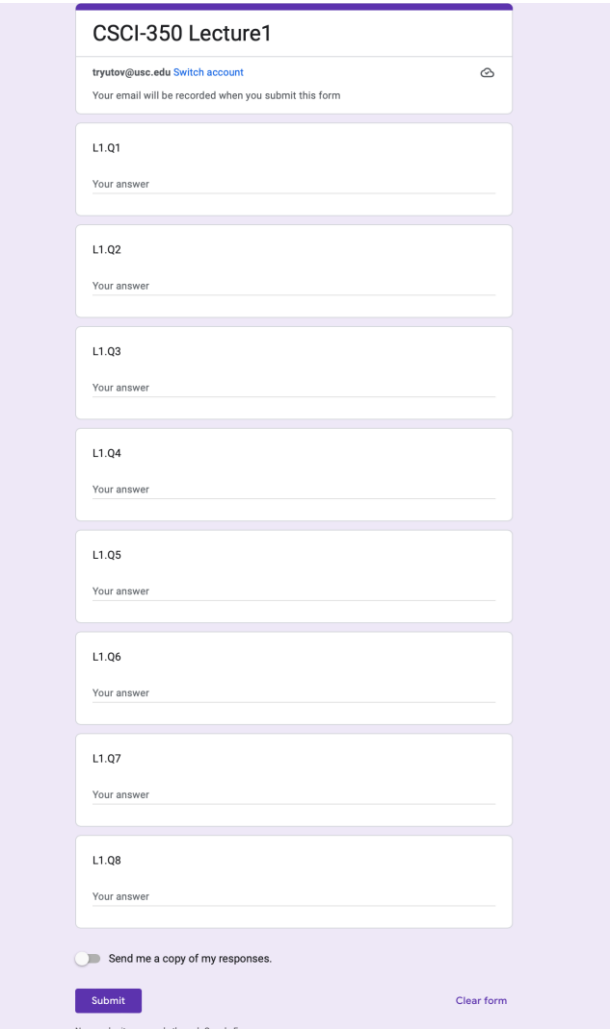
Class Participation: Discussion Questions

- Logistics:
  - We will use Google forms to collect responses for in-lecture discussion questions
    - One form per lecture
  - Response submission link for each lecture will be posted on Piazza in Resources
    - The link will be active for 72 hours
  - There will be about 8 questions during each lecture
    - You will have about 1-5 minutes to answer each question
    - You will use the same form to submit each question
- Purpose:
  1. Get you engaged with the lecture material
  2. Promote active learning
  3. Provide feedback for me (e.g., what topics to review)
  4. Keep track of your learning progress
- The discussion questions will not be included in lecture notes and will not be posted

© Tatyana Ryutov

Where can I find the Google form?





USC Viterbi School of Engineering

© Tatyana Ryutov

Class Participation: Assessment

- In **the end of the** semester we will count your relevant answers to the questions asked to compute your class participation score
- We will drop all answers that are unrelated to the questions
- We know you have access to the questions from previous semesters:
  - While some questions in this semester can be similar to those from the previous ones, the order of the questions will be different
  - Even if you answer is admissible but it appears in the wrong position, it will not be counted
- **It is your responsibility to keep track of how many questions you answered! We will not check it for you**

- 5% of your total grade:
  - If you answer ≥ 90% off all questions asked (not just forms) you get 5%
  - If you answer ≥ 80% < 90% off all questions asked you get 4%
  - If you answer ≥ 70% < 80% off all questions asked you get 3%
  - If you answer ≥ 60% < 70% off all questions asked you get 2%
  - If you answer ≥ 50% < 60% off all questions asked you get 1%
  - If you answer < 50% off all questions asked you get 0%

USC Viterbi School of Engineering

© Tatyana Ryutov

Be aware of assignments...

- Start early
  - These are NOT 2-3 hour assignments
- Pay attention to the discussions on Piazza
- Use GitHub properly
  - You don’t want to lose work
  - You want to be able to branch or rebase
- Plan your coding
  - Have a reliable partner, if any

© Tatyana Ryutov

Grace Days

- Programming Assignments have due dates and times
  - Due on the given dates at **11:59 PM Pacific**
    - The following day at 12:01 AM is late
- You are allowed five grace days: 5 extensions of 24 hours each
  - If you are using one or more grace days, you need to let us know
    - You will fill in a form, see Piazza
  - If you request additional grace days from the instructor, you must have a documented reason for each grace day used to accompany your request
  - Once you have used your grace days, any late submission will not be accepted

USC Viterbi School of Engineering

© Tatyana Ryutov

What won’t get additional grace days?

- Starting late
- Falling behind in general
- Difficulty of programming
- Work or exams for other classes
- Interview with a potential employer
- Forgot to push to GitHub
- Your clock disagreeing with ours on what 11:59 PM is
  - Do not cut it too close
  - Leave time for things to “go wrong”
- This is not an exhaustive list!

© Tatyana Ryutov

Academic Honesty

- Academic integrity is taken very seriously
  - It can end your career
  - No work will be considered to have been submitted until you have completed the class rules quiz
- **NO AI tools are allowed in this class!!!**
  - Using something like ChatGPT to complete an assignment/exam is plagiarism
- The university-wide academic honesty rules also apply
  - Resource for USC academic standards:
  - https://viterbischool.usc.edu/academic-integrity/
- We will be running Measure Of Software Similarity (MOSS) on some submissions
- We will be reporting any infractions suspected
  - Suggested penalty is F for the class

Hard work leads to self-fulfillment and gratification, not just a good grade!

USC Viterbi School of Engineering

© Tatyana Ryutov

CSCI 350 Supplement to University Rules

- For details, see rules posted on Piazza
- Some highlights:
  - Any code obtained with help of TA or CP must be clearly marked in comments with who and when
  - The help you receive from classmates should be limited to conceptual
  - If you receive help from others, follow the rule:
    - Everything that you derive from the collaboration should be in your head
    - After the meeting, engage in at least a half-hour of mind-numbing activity
    - Goal: do not copy each other
  - You can seek help with C language and xv6

You are explicitly prohibited from seeking help outside of CSCI 350 resources for the solutions to the programming projects

USC Viterbi School of Engineering

© Tatyana Ryutov

Programming Projects Rules for Groups

- Group size is limited to 2
  - Piazza is a good place for finding a partner
  - You are not required to have a partner
- Fill in partner sign up form
  - You still need to sign up if you are working on your own
- You may not split once work on a project has begun
  - Alternately, if you do, neither of you may use any common code
- Only one join and one split per person is allowed during the semester
- For each function written, or skeleton code fragment modified, clearly mark with comments which group member(s) wrote or modified the code
  - Be sure to include the last (family) name of each person in such a comment

USC Viterbi School of Engineering

© Tatyana Ryutov

Supplement for Groups (continued)

- Groups are required to use GitHub via private repository to track commits
- Each student is expected to pull his/her own weight
- It is the responsibility of each student that the submission bearing his/her name abides by the academic honesty rules
- No credit will be given for any subset of a project that fails to meet academic integrity standards
  - Even if that portion is determined to be the fault of only one group member

USC Viterbi School of Engineering

© Tatyana Ryutov

Team Or No Team?

- Team
  - Allows for brainstorming and mutual support
  - Mimics industrial setting, e.g., use of shared source code repository and versioning
- No team
  - Your life may be easier
  - You do not get more credit if you work alone

© Tatyana Ryutov

USC Viterbi School of Engineering

Learning Objectives

- Understand the basic organization of operating systems (OS)
- Understand some of the core abstractions most OSs support, such as address space, processes, thread, file
- Understand the basic services OS provides:
  - Process management
  - CPU scheduling
  - Memory management
  - Storage management
  - Protection and security
- Analyze design choices and trade-offs used in the implementation of OS components
- Understand basic design principles of secure (trusted) OS

Theory

- Develop new OS functionality related to:
  - System calls
  - Thread management
  - Process synchronization
  - Process scheduling
  - Memory management
- Read, interpret, and evaluate OS code that you did not write
- Modify and debug existing operating system code to achieve the desired OS functionality
- Practice techniques related to distributed parallel computing (map-reduce)

Hands-on

USC Viterbi School of Engineering

© Tatyana Ryutov

Learning Objectives and Course Team Responsibilities

- Understand the basic organization of operating systems (OS)
- Understand some of the core abstractions most OSs support, such as address space, processes, thread, file
- Understand the basic services OS provides:
  - Process management
  - CPU scheduling
  - Memory management
  - Storage management
  - Protection and security
- Analyze design choices and trade-offs used in the implementation of OS components
- Understand basic design principles of secure (trusted) OS
- Develop new OS functionality related to:
  - System calls
  - Thread management
  - Process synchronization
  - Process scheduling
  - Memory management
- Read, interpret, and evaluate OS code that you did not write
- Modify and debug existing operating system code to achieve the desired OS functionality
- Practice techniques related to distributed parallel computing (map-reduce)

Instructor

TAs, CPs

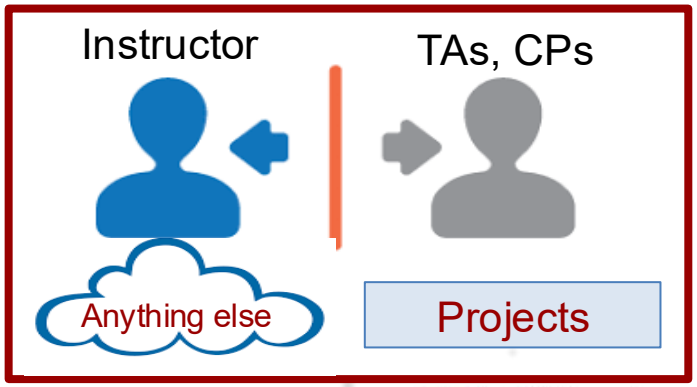
USC Viterbi School of Engineering

© Tatyana Ryutov

How to get help?

- Programming projects – TA and CPs
  - TAs are available throughout the week
  - Resource for students for project assignments
    - Won't typically be able to spend the time to debug everyone's assignment, but can
      - Help with the understanding
      - Help with the concepts
      - Help with the syntax
      - Help you understand what your code is doing
  - See TA and CPs OH schedule on Piazza
- Anything else – instructor
- Piazza
  - Do not post your code there!

The course team is available to answer your email/Piazza questions 9:00am-6:00pm on weekdays only
No office hours during University holidays and Fall recess



USC Viterbi School of Engineering

© Tatyana Ryutov

Note

- Don’t expect 24x7 answers
- Try to figure out yourself
- Piazza is not designed for debugging
- Utilize right venue: go to office hours
- Send detailed questions/bug reports, not “no idea what’s wrong”
- Instructor/CPs/Graders are not on pager duty 24 x 7
- Don’t expect response before next business day
- Implications:
  - Students should answer each other
  - Start your assignments early!

© Tatyana Ryutov

xv6 projects

- Why do a “real” OS kernel?
  - Deeper understand some OS concepts studied in class
  - Lean to apply general OS concepts to specific system
  - Get a hands-on experience with the kernel implementation
  - Improve your programming skills
  - Learn how to debug
- Project themes:
  1. System calls
  2. Threads management, synchronization
  3. CPU scheduling (optional)
  4. Memory management (optional)
  5. Map-reduce (not xv6 project)
- Get started now!
  - See Piazza for resources

© Tatyana Ryutov

Projects and Lectures

- How can we apply lecture material to our projects? Will the lectures and projects cover the same topics?

Project1

Project2

Project3

Project4

Project5

Week	Topic	Readings
8/29 Lec1	Introduction Process abstraction System calls intro	T1: Chapters 1.1; 1.2 T1: Chapter 2.1 T2: Chapters 3.1.-3.1.3 T2: Chapter 2.3
9/5 Lec2	Dual mode operation Context switch System call implementation	T1: Chapters 2.2; 2.3; T1: Chapters 2.4.1-2.4.5 T1: Chapter 2.6
9/12 Lec3	Kernel organization Virtual machines Process management Interrupts	T2: Chapters 2.8.1-2.8.3 T2: Chapter 18 T1: Chapters 3.1-3.3 T2: Chapter 1.2.1 T1: Chapters 2.4.4; 2.4.5; 2.5
9/19 Lec4	Concurrency and threads	T1: Chapters 4.1-4.8
9/26 Lec5	Synchronization	T1: Chapters 5.1-5.5; 5.7; 5.8
10/17 Lec6	CPU scheduling	T1: Chapter 7.1
10/24 Lec7	Advanced Synchronization Memory Management	T1: Chapter 6.5 T2: Chapter 8 T1: Chapters 8.1-8.3
10/31 Lec8	Storage Devices File System implementation	T1: Chapter 12 T1: Chapters 11; 13.1- 13.3.3
11/7 Lec9	COW File Systems Reliable Storage Distributed Systems intro	T1: Chapter 13.3.4 T1: Chapter 14 T2: Chapters 19.1-19.3
11/14 Lec10	Remote Procedure Call Distributed Mutual exclusion Distributed parallel computing Map-reduce computing paradigm	T3: Chapter 4.2 T3: Chapter 6.3 MapReduce paper
12/5 Lec11	Protection and Security Trusted OS design	T2: Chapters 14.4-14.6; 15.1-15.3; 15.6-15.8

USC Viterbi School of Engineering

© Tatyana Ryutov

What is xv6?

- xv6 is an MIT educational reimplementation of Unix v6
  - Even smaller than v6
  - Runs with multiple processors/multicore
  - Supports paging, processes, files, etc.
- Runs on x86 (to be precise on QEMU)
  - QEMU (short for Quick Emulator) is a generic open-source machine emulator
- Installation instructions: see Project1 on Piazza
- Why study xv6 instead of Linux, Solaris, or Windows?
  - Big enough to illustrate basic OS concepts and implementation
  - Small enough to be (relatively) easy to understand
    - A subset of Unix system calls, e.g., fork, exec, read, write, …
  - Similar enough
    - To modern OSs
    - Once you've explored xv6, you'll find your way inside kernels such as Linux

© Tatyana Ryutov

xv6 Overview

- xv6 is a monolithic, preemptively-multitasked, multiprocessor-capable, 32-bit, UNIX-like operating system
- Some limitations:
  - Max addressable memory: 2GB
  - Few supported devices (e.g., no network)
  - No support for kernel-level threading
- Very limited set of user-level programs: shell, cat, echo, grep, kill, ln, ls, mkdir, rm, wc
- xv6 runs on an x86 (Intel) processor, and relies on many of its hardware features e.g., privilege levels (kernel/user mode), interrupt vector, etc.

© Tatyana Ryutov

Understanding xv6

- Lectures (general concepts) + project presentations (detailed project discussions) + study code on your own (read xv6 textbook and code) + programming assignments + free response questions

- Resources:
  - getting started tips:
  - http://pdos.csail.mit.edu/6.828/2010/overview.html
  - xv6 has a very useful guide that will greatly assist you throughout the course assignments: http://pdos.csail.mit.edu/6.828/2011/xv6/book-rev6.pdf
  - You may also find the following useful: http://pdos.csail.mit.edu/6.828/2011/xv6/xv6-rev6.pdf

Start learning it now!

USC Viterbi School of Engineering

© Tatyana Ryutov

xv6 files

- **Generic**: asm.h (segmentation), mmu.h, x86.h (inline assembly), elf.h (ELF format), types.h, param.h (kernel constants), string.c
- **Boot**: bootasm.S, bootother.S, bootmain.c, main.c
- Process and virtual memory: proc.h, proc.c, vm.c, zombie.c, pipe.c, exec.c, kalloc.c, sysproc.c, swtch.S, initcode.S
- **System call and interrupt**: syscall.h, traps.h, trap.c, syscall.c, trapasm.S, vector.S
- **Synchronization and multicore**: spinlock.h, mp.h, spinlock.c, mp.c
- **Disk and file system**: defs.h, fs.h, stat.h file.h, buf.h, fcntl.h, bio.c, fs.c, file.c, sysfile.c
- **Device**: kbd.h, kbd.c, timer.c, lapic.c, picirq.c, uart.c, console.c, ide.c, ioapic.c
- **User-mode programs**: user.h, sh.c, forktest.c, wc.c, kill.c, cat.c, grep.c, stressfs.c, ln.c, ulib.c, echo.c, init.c, ls.c, printf.c, umalloc.c, usertests.c, mkdir.c, rm.c, usys.S
- **Other**: mkfs.c, Makefile

You do not need to remember this for the exams

© Tatyana Ryutov

Students' Questions

- Why we do not study xv6 in class?
- Why we do not discuss projects in class?

We simply will not have enough time!

- If you understand the material discussed in class, you should be able to learn xv6 yourself
- There will be project discussions (but not during the lecture time)

© Tatyana Ryutov

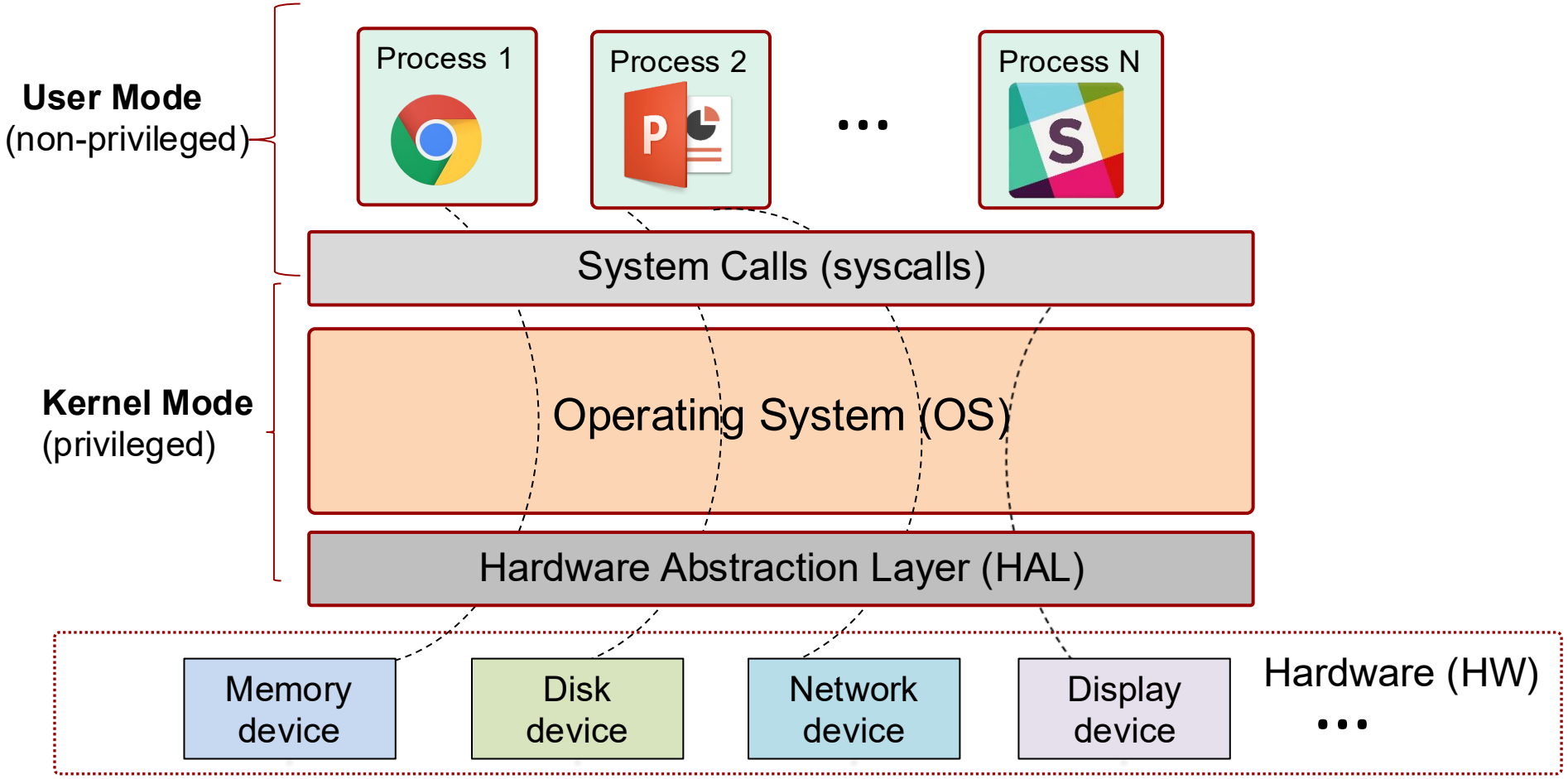
Lecture Outline

- Course information
- What is OS?
- Address space abstraction
- Process abstraction
- Syscall intro

© Tatyana Ryutov

What is Operating System (OS)?

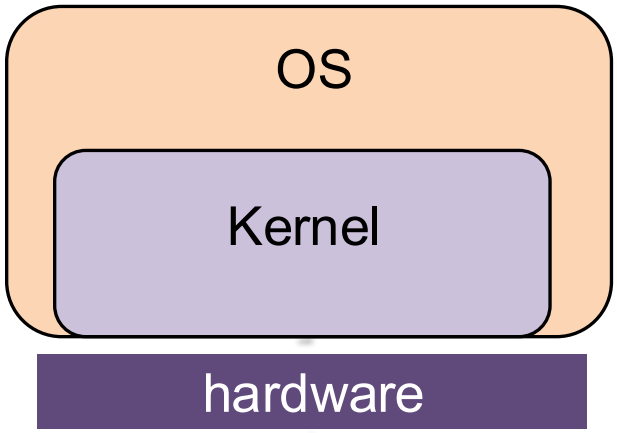
- Software to manage computer resources for its users and applications



© Tatyana Ryutov

What is the OS Kernel?

- Core component of OS
  - Has full access to all of the HW
- Responsible for:
  - Process management
  - CPU scheduling
  - Memory management
  - Storage management
  - Protection and security
  - Device and I/O management (not discussed in class)
- The focus of this class is on the kernel!



© Tatyana Ryutov

How will we learn the kernel?

Core kernel functions:

- Process management
- CPU scheduling
- Memory management
- Storage management
- Protection and security

Week	Topic	Readings
8/29 Lec1	Introduction Process abstraction System calls intro	T1: Chapters 1.1; 1.2 T1: Chapter 2.1 T2: Chapters 3.1.-3.1.3 T2: Chapter 2.3
9/5 Lec2	Dual mode operation Context switch System call implementation	T1: Chapters 2.2; 2.3; T1: Chapters 2.4.1-2.4.5 T1: Chapter 2.6
9/12 Lec3	Kernel organization Virtual machines Process management Interrupts	T2: Chapters 2.8.1-2.8.3 T2: Chapter 18 T1: Chapters 3.1-3.3 T2: Chapter 1.2.1 T1: Chapters 2.4.4; 2.4.5; 2.5
9/19 Lec4	Concurrency and threads	T1: Chapters 4.1-4.8
9/26 Lec5	Synchronization	T1: Chapters 5.1-5.5; 5.7; 5.8
10/17 Lec6	CPU scheduling	T1: Chapter 7.1
10/24 Lec7	Advanced Synchronization Memory Management	T1: Chapter 6.5 T2: Chapter 8 T1: Chapters 8.1-8.3
10/31 Lec8	Storage Devices File System implementation	T1: Chapter 12 T1: Chapters 11; 13.1- 13.3.3
11/7 Lec9	COW File Systems Reliable Storage Distributed Systems intro	T1: Chapter 13.3.4 T1: Chapter 14 T2: Chapters 19.1-19.3
11/14 Lec10	Remote Procedure Call Distributed Mutual exclusion Distributed parallel computing Map-reduce computing paradigm	T3: Chapter 4.2 T3: Chapter 6.3 MapReduce paper
12/5 Lec11	Protection and Security Trusted OS design	T2: Chapters 14.4-14.6; 15.1-15.3; 15.6-15.8

Topics is red are required by ABET accreditation*, se we have to learn those

*ABET accreditation ensures that graduates have met the educational requirements necessary to enter the profession

© Tatyana Ryutov

USC Viterbi School of Engineering

OS Challenges

- Reliability
  - Does the system do what it was designed to do?
- Availability
  - What portion of the time is the system working?
- Security
  - Can the system be compromised by an attacker?
- Performance
  - Latency (or response time)
    - How long does an operation take to complete?
  - Throughput
    - How many operations can be done per unit of time?
  - Overhead
    - How much extra work is done by the OS?
  - Fairness
    - How equal is the performance received by different users?
  - Predictability
    - How consistent is the performance over time?
- Portability & adoption

USC Viterbi School of Engineering

© Tatyana Ryutov

OS is a Complex Mess

- How can we build and understand a complex, messy system?
  - Come up with some structure
    1. Manage complexity through abstraction (e.g., address space, process, file)
    2. Apply good design principles (e.g., separation of policy from mechanism)
    3. Consider tradeoffs (e.g., optimize for common case)

© Tatyana Ryutov

Lecture Outline

- Course information
- What is OS?
- Address space abstraction
- Process abstraction
- Syscall intro

© Tatyana Ryutov

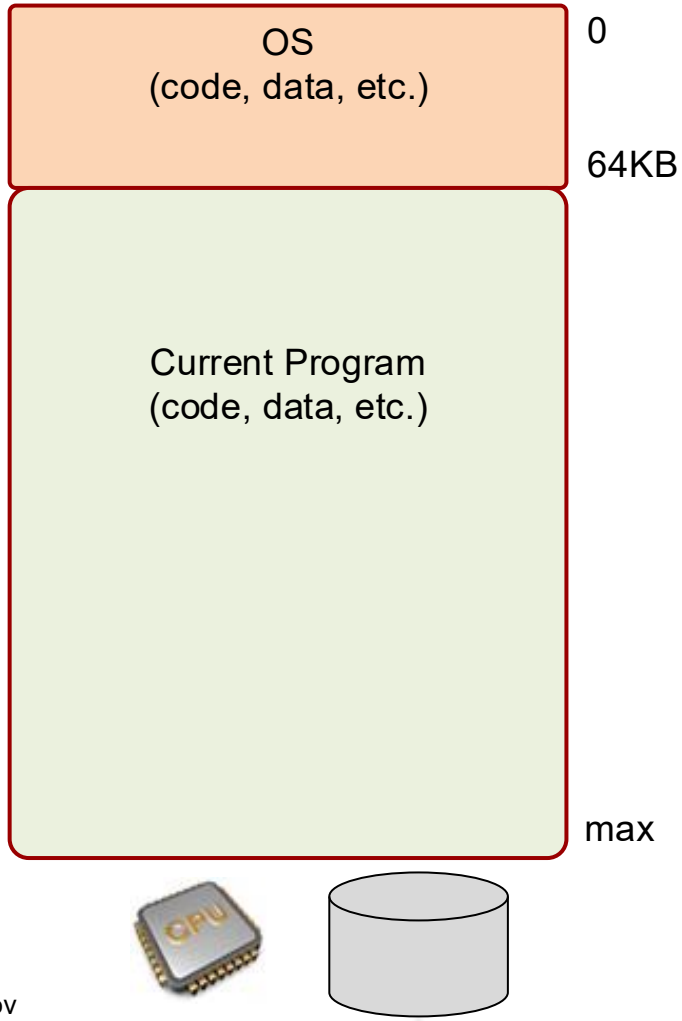
Note

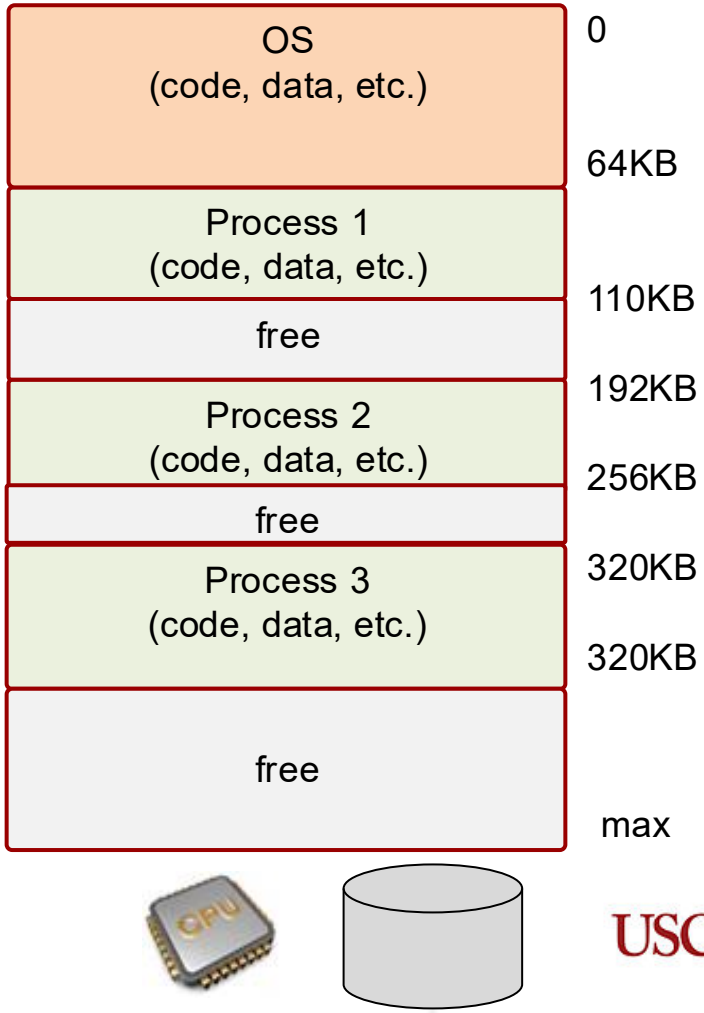
- Today we will see just basics of foundational concepts, such as address space, process abstraction, user/kernel mode, syscall processing
- We will keep revisiting these concepts in following lectures to deepen your understanding

© Tatyana Ryutov

Lack of Memory Abstraction

- Early systems did not provide memory abstraction to users
- Time-sharing demanded OS to implement memory sharing efficiently



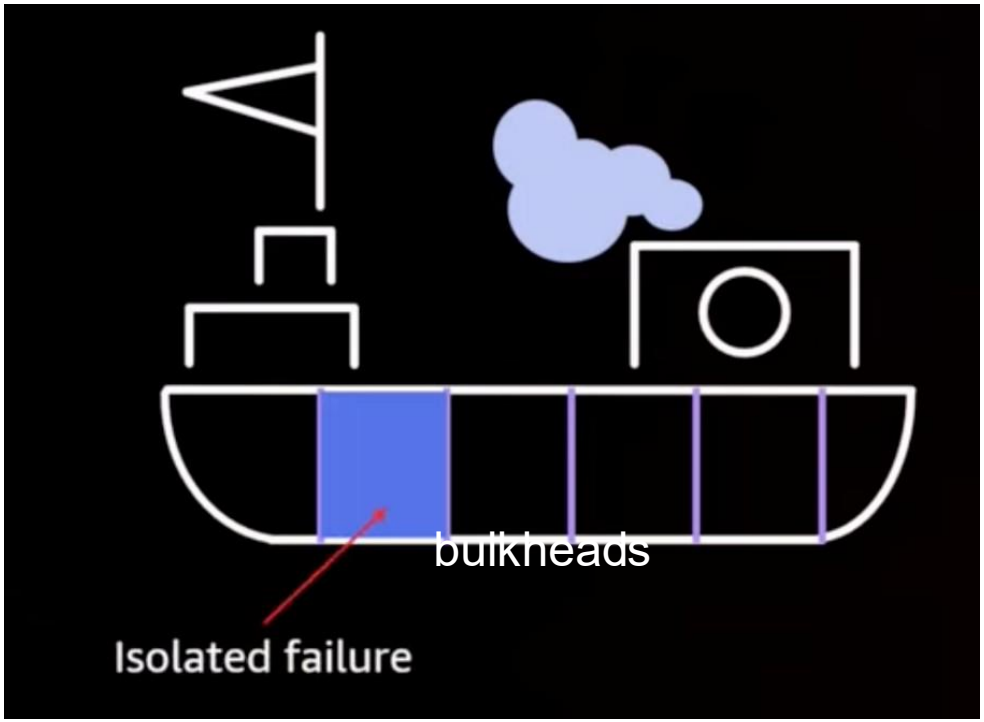


© Tatyana Ryutov

USC Viterbi School of Engineering

Sidenote: The Principle of Isolation

- Isolation is a key principle in building reliable and secure systems
- If two entities are properly isolated from one another, one can fail without affecting the other

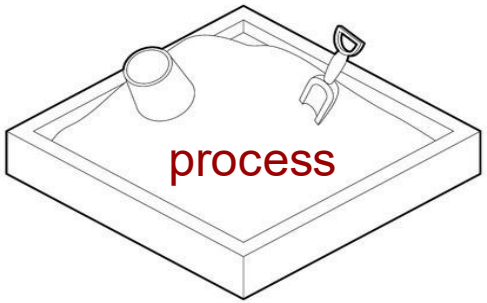


© Tatyana Ryutov

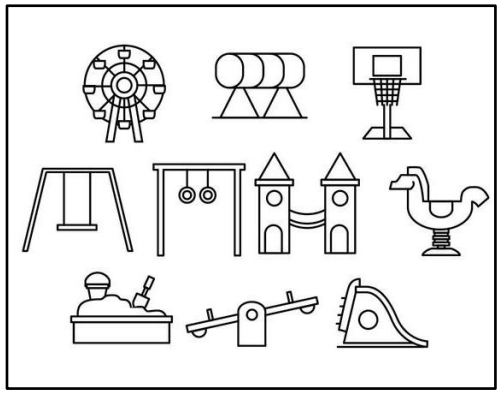
USC Viterbi School of Engineering

Address Space Abstraction

- Abstraction of physical memory
  - Running program's view of physical memory
- Goals:
  - Give each process a private memory area
  - Prevent one process from reading/writing outside its private memory
  - Allow sharing, when needed
- Each process has it's own address space (user space)
- OS has its own address space (kernel space)



virtual address space
mapped to physical memory



OS

© Tatyana Ryutov

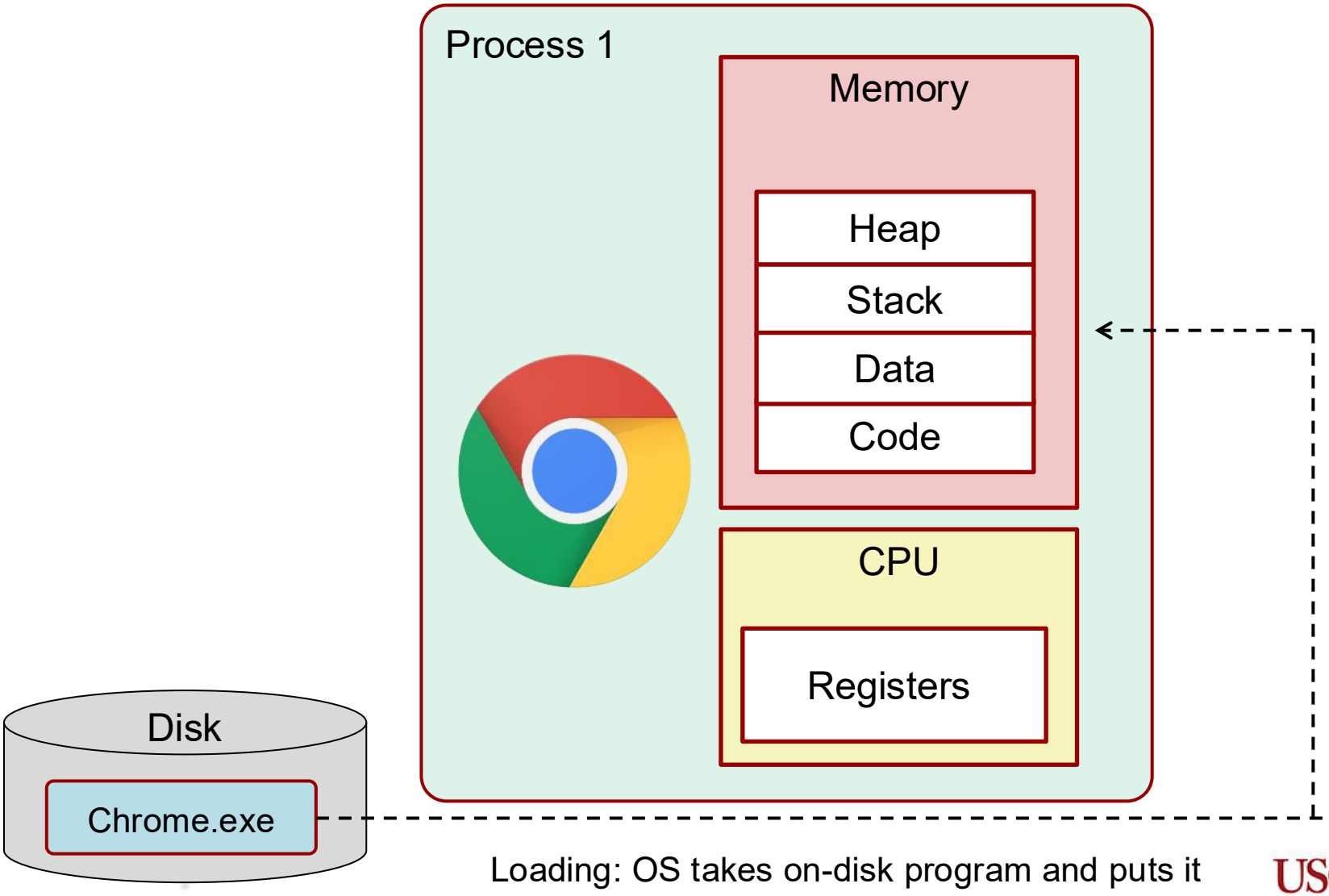
USC Viterbi School of Engineering

Lecture Outline

- Course information
- What is OS?
- Address space abstraction
- Process abstraction
- Syscall intro

© Tatyana Ryutov

What is a process?

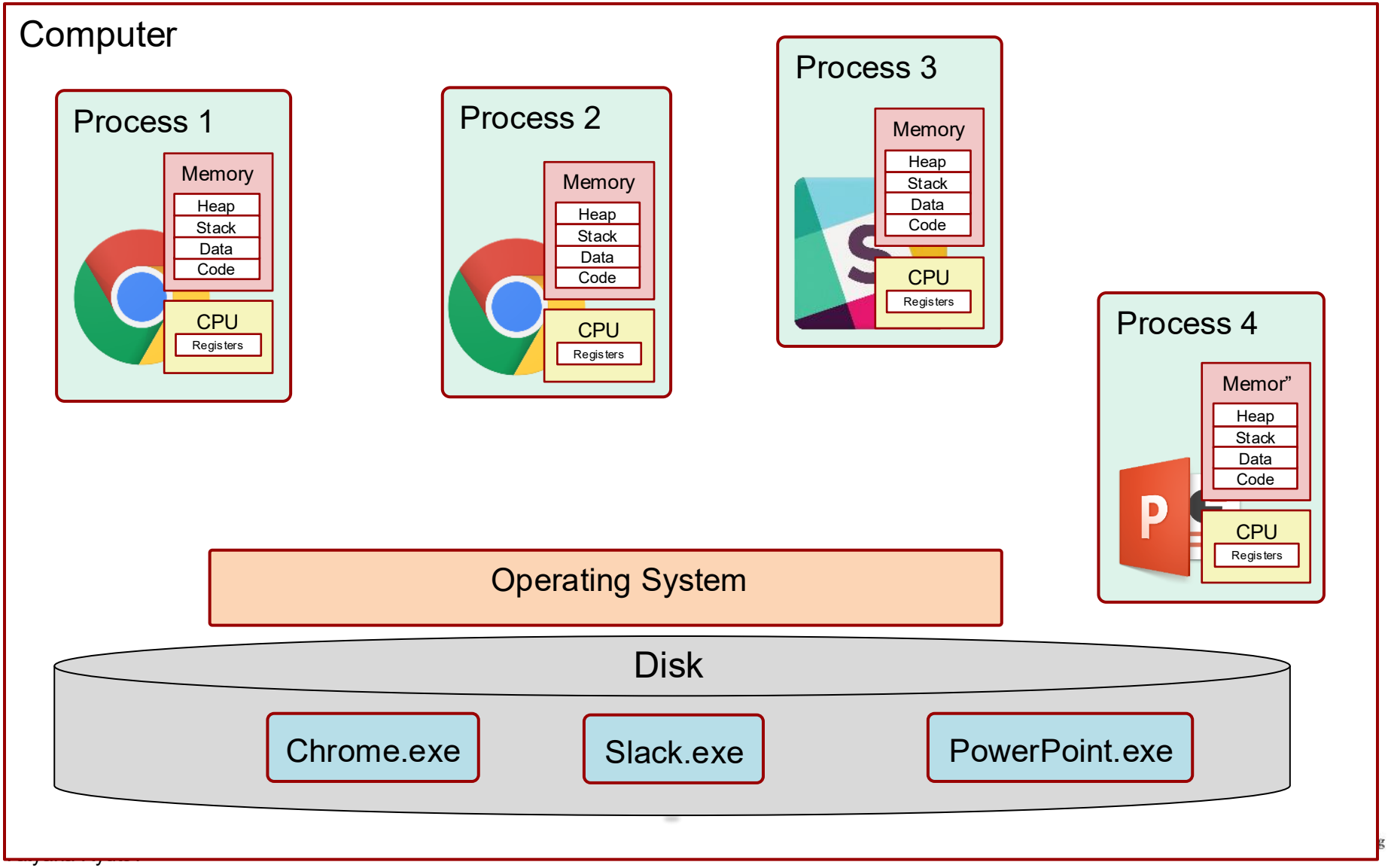


Loading: OS takes on-disk program and puts it into the process address space

© Tatyana Ryutov

USC Viterbi School of Engineering

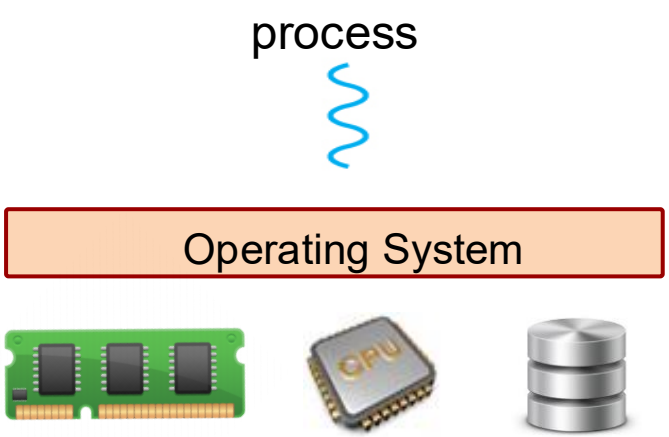
What is a process?



©

Process Abstraction

- Process ≠ Program
- Process is an instance of a program, running with limited rights
- Process consists of two parts:
  1. Address space – memory for running
  2. Process Control Block (PCB) – info about each process

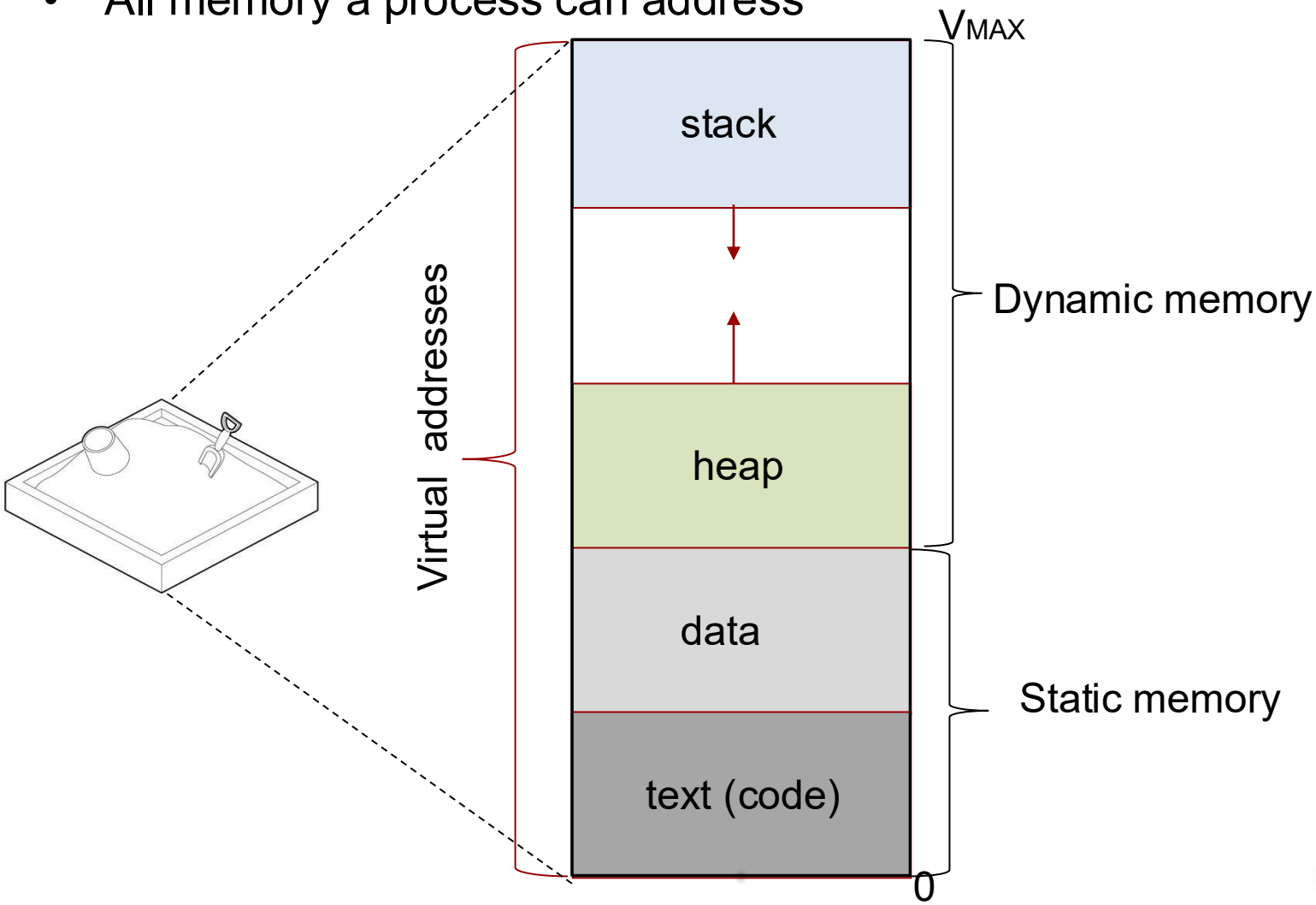


© Tatyana Ryutov

USC Viterbi School of Engineering

Process Address Space

- All memory a process can address



© Tatyana Ryutov

USC Viterbi School of Engineering

Example

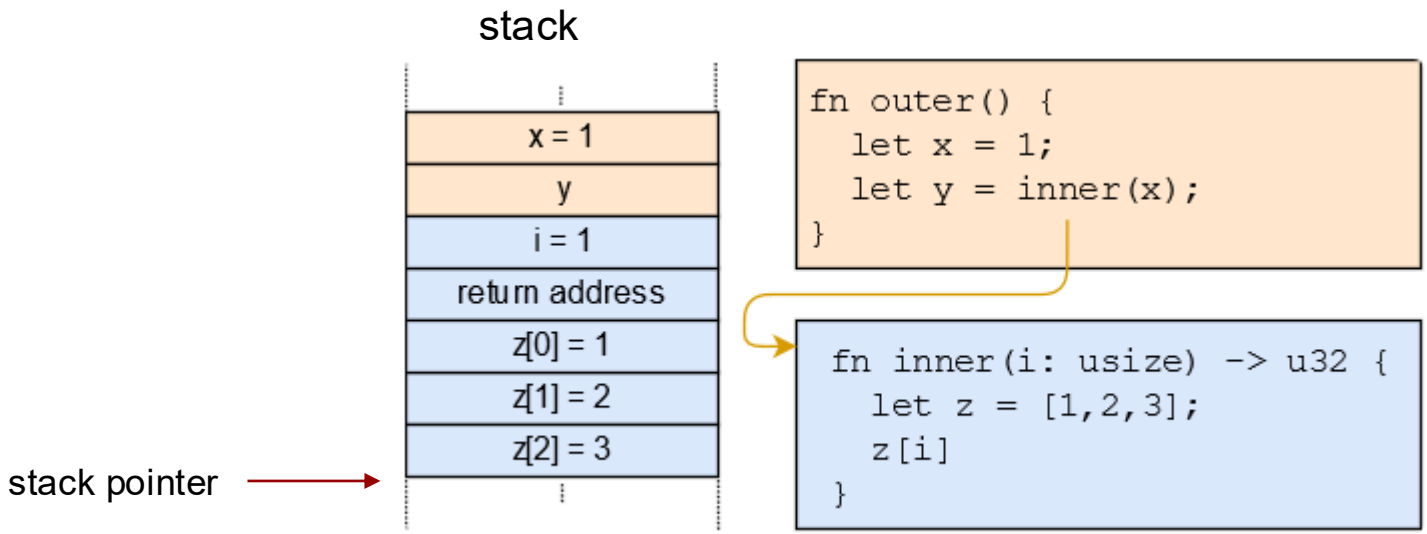
```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
   printf("location of code : %p\n", main);
   printf("location of heap : %p\n", malloc(100e6));
   int x = 3;
   printf("location of stack: %p\n", &x);
   return x;
}
```

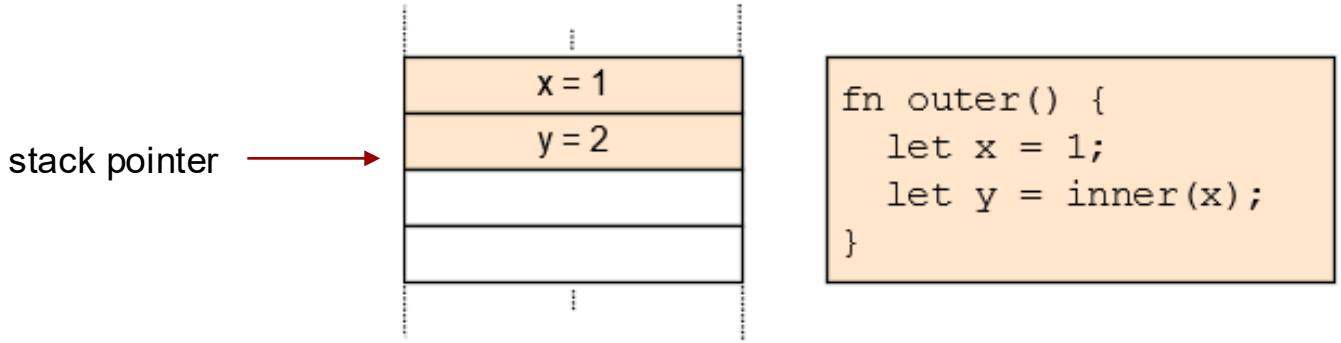
When run on a 64-bit Mac, we get the following output:

location of code : 0x1095afe50
location of heap : 0x1096008c0
location of stack: 0x7fff691aea64

© Tatyana Ryutov

Example: Function Invocation

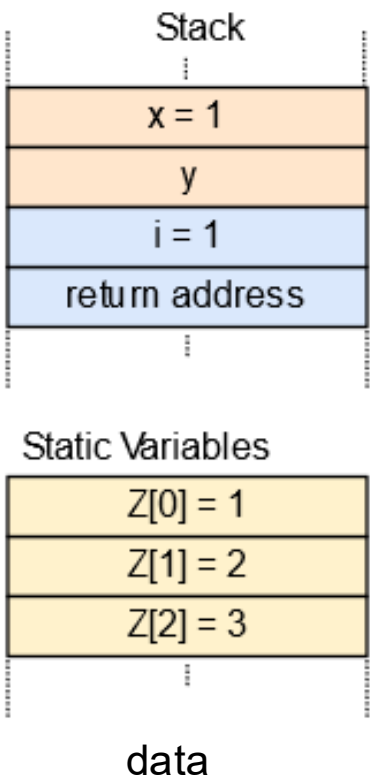




© Tatyana Ryutov

USC Viterbi School of Engineering

Example: Static Variables



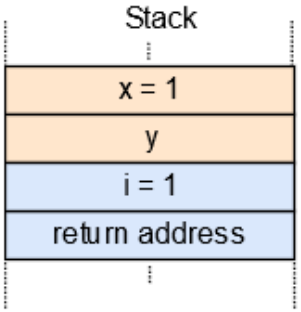

```
fn outer() {
  let x = 1;
  let y = inner(x);
}
```

```
static Z: [u32; 3] = [1,2,3];
```

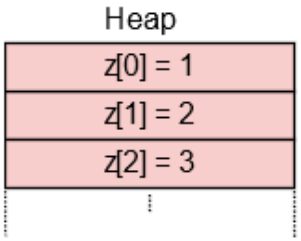
```
fn inner(i: usize) -> &'static u32 {
  &Z[i]
}
```

© Tatyana Ryutov

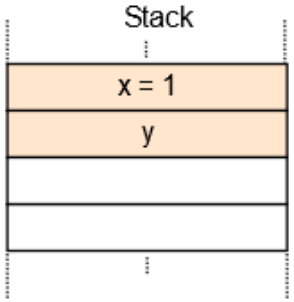
Example: Dynamic Memory



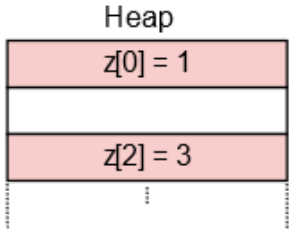

```
fn outer() {
  let x = 1;
  let y = inner(x);
  deallocate(y, size_of(u32));
}
```




```
fn inner(i: usize) -> *mut u32 {
  let z = allocate(size_of([u32;3]));
  z.write([1,2,3]);
  (z as *mut u32).offset(i)
}
```



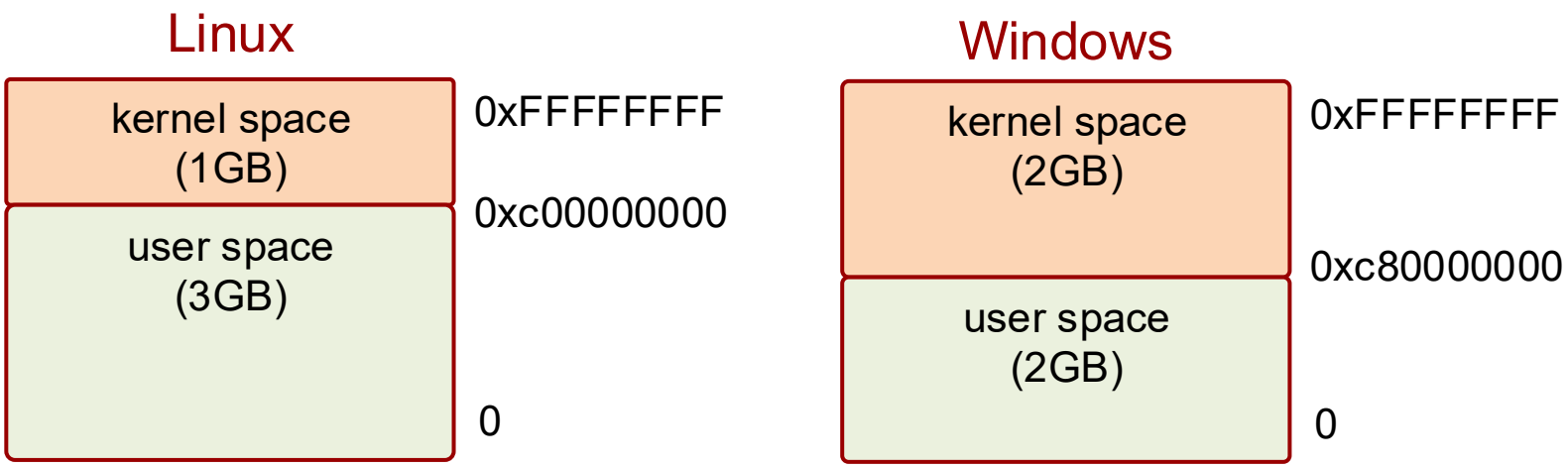

```
fn outer() {
  let x = 1;
  let y = inner(x);
  deallocate(y, size_of(u32));
}
```



© Tatyana Ryutov

Address Space: Implementation

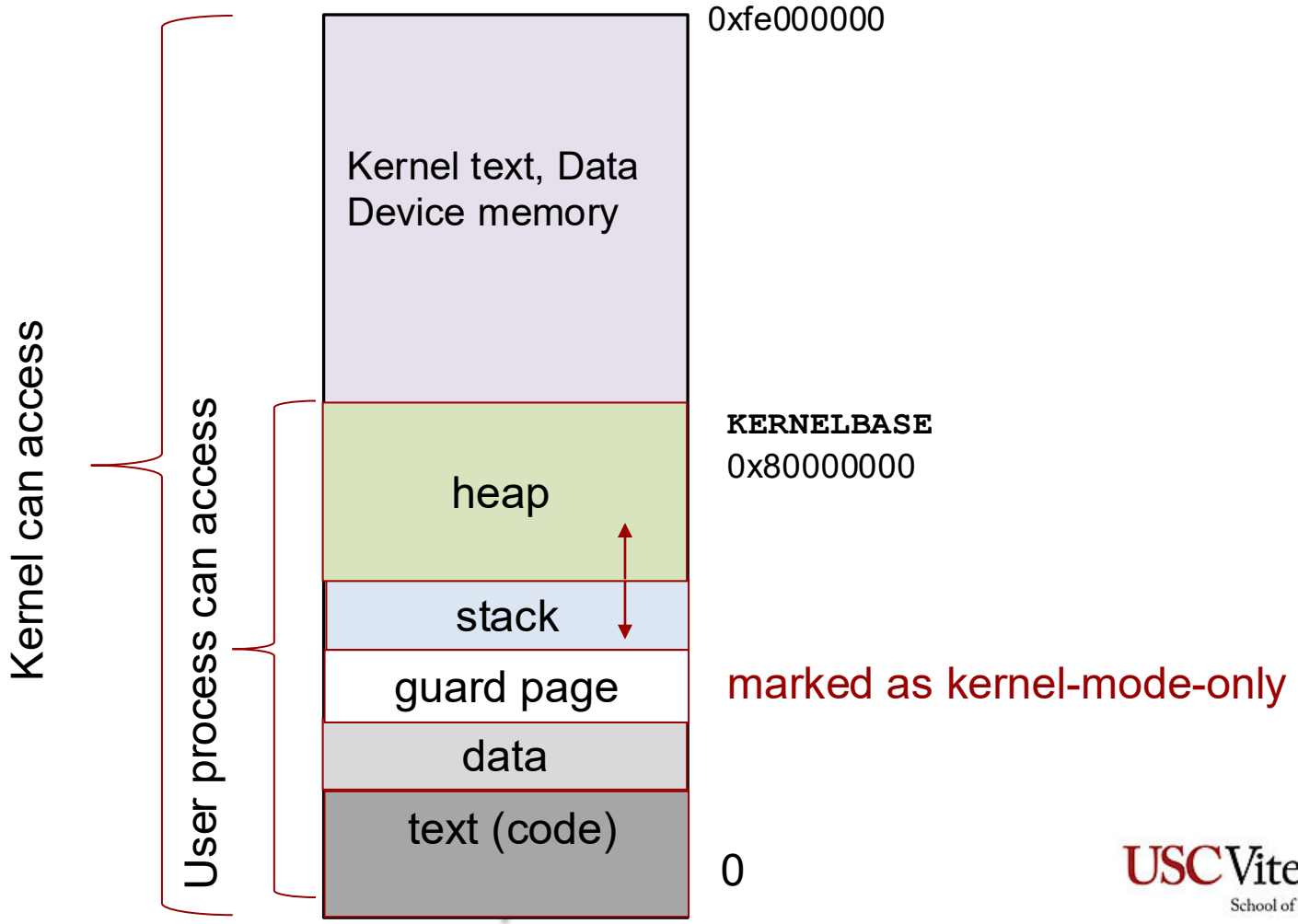
- Usually implementation is split between OS and HW
  - OS manages address spaces
  - HW performs virtual→physical address translation and protection
  - Why split?



© Tatyana Ryutov

Process Address Space in xv6

- Process's user memory starts at virtual address zero and can grow up to **`KERNBASE`**, allowing a process to address up to 2 GB of memory

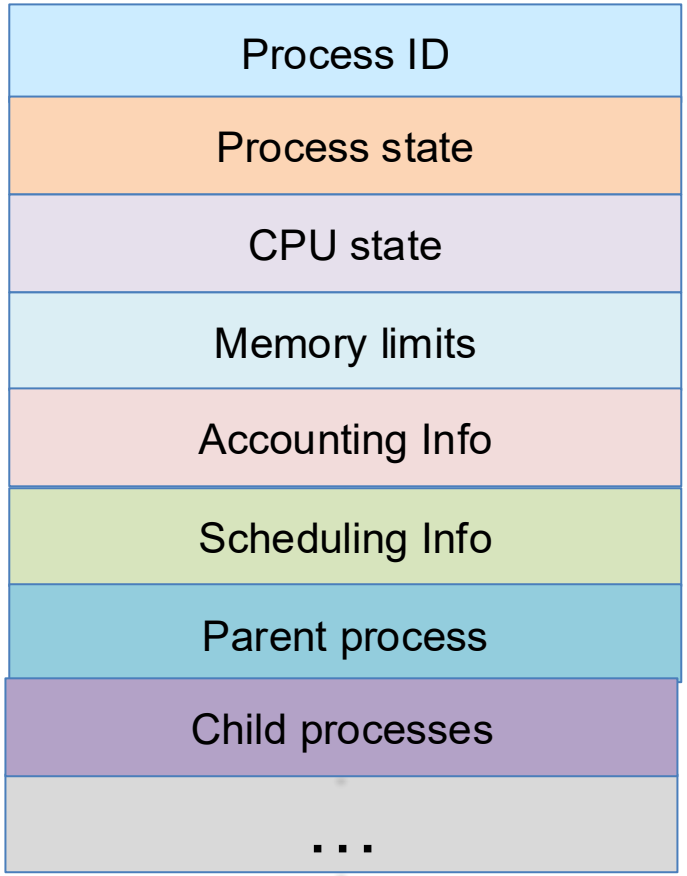


© Tatyana Ryutov

USC Viterbi School of Engineering

Process Control Block (PCB)

- OS maintains information about every process in a data structure called Process Control Block (PCB)
- PCB contains:
  - Unique process identifier
  - Process state (Running, Ready, etc.)
  - CPU state (program counter, registers, flags)
  - Memory management information (e.g., page tables, segment tales, base & bound registers)
  - CPU scheduling & accounting information
  - Parent process
  - Child processes
  - ...
- PCB is known by different names in different OS
  - `struct proc` in xv6
  - `task_struct` in Linux



© Tatyana Ryutov

PCB in xv6: `struct proc`

```
// Per-process state
struct proc {
  uint sz;                     // Size of process memory (bytes)
  pde_t* pgdir;                // Page table
  char *kstack;                // Bottom of kernel stack for this process
  enum procstate state;        // Process state
  int pid;                     // Process ID
  struct proc *parent;         // Parent process
  struct trapframe *tf;        // Trap frame for current syscall
  struct context *context;     // swtch() here to run process
  void *chan;                  // If non-zero, sleeping on chan
  int killed;                  // If non-zero, have been killed
  struct file *ofile[NOFILE]; // Open files
  struct inode *cwd;           // Current directory
  char name[16];               // Process name (debugging)
};
```

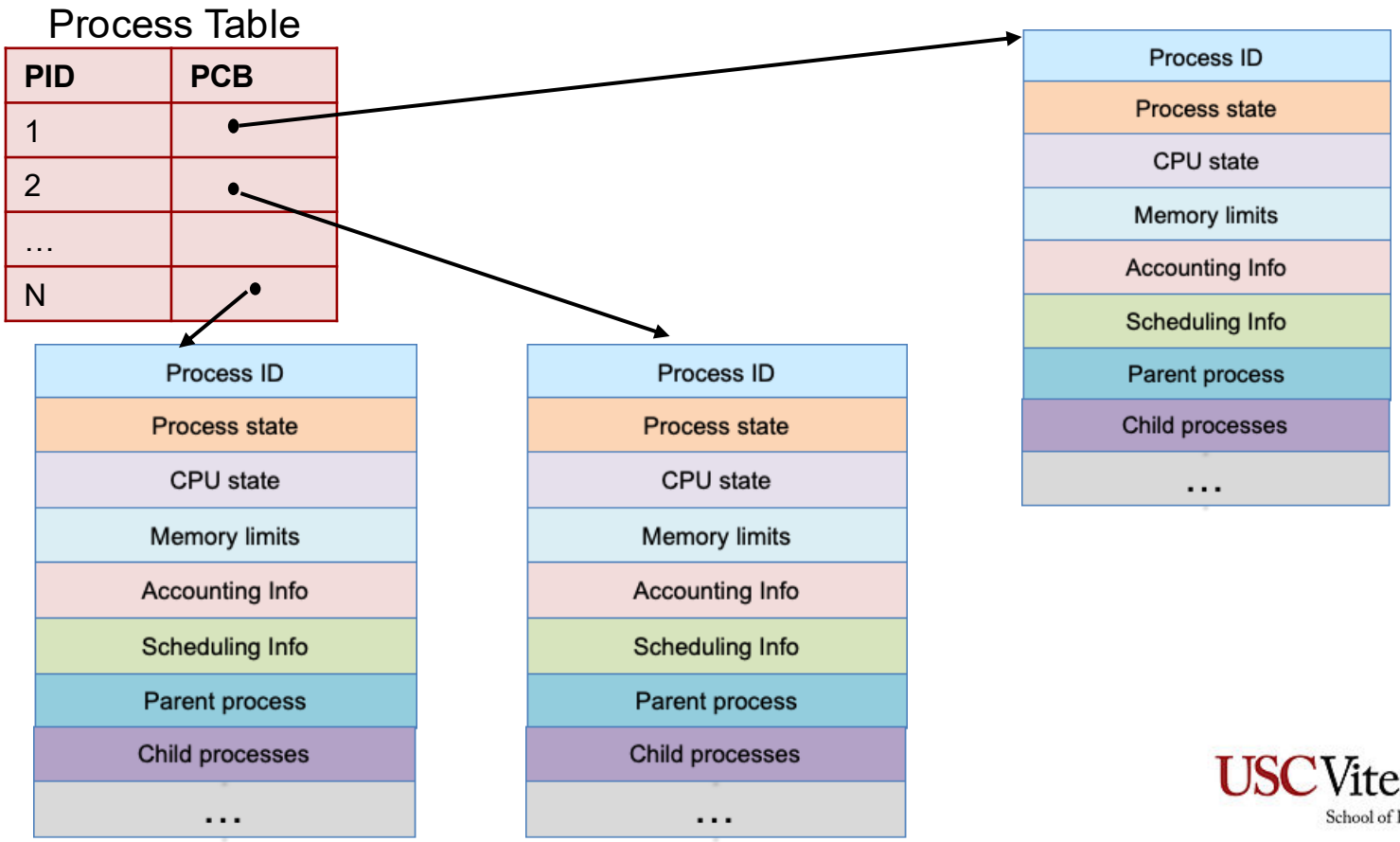
proc.h

USC Viterbi School of Engineering

© Tatyana Ryutov

PCBs of Multiple Processes

- PCB table - set of PCBs of all active processes is a kernel data structure



© Tatyana Ryutov

USC Viterbi School of Engineering

PCB Table (`ptable`) in xv6

- Fixed-size array of all processes
- Global variable that is loaded into memory during the booting of xv6 kernel
  - Real kernels have dynamic-sized data structures

```
struct {
   struct spinlock loc;
   struct proc proc[NPROC];
} ptable;
```

`NPROC` is initialized to 64

© Tatyana Ryutov

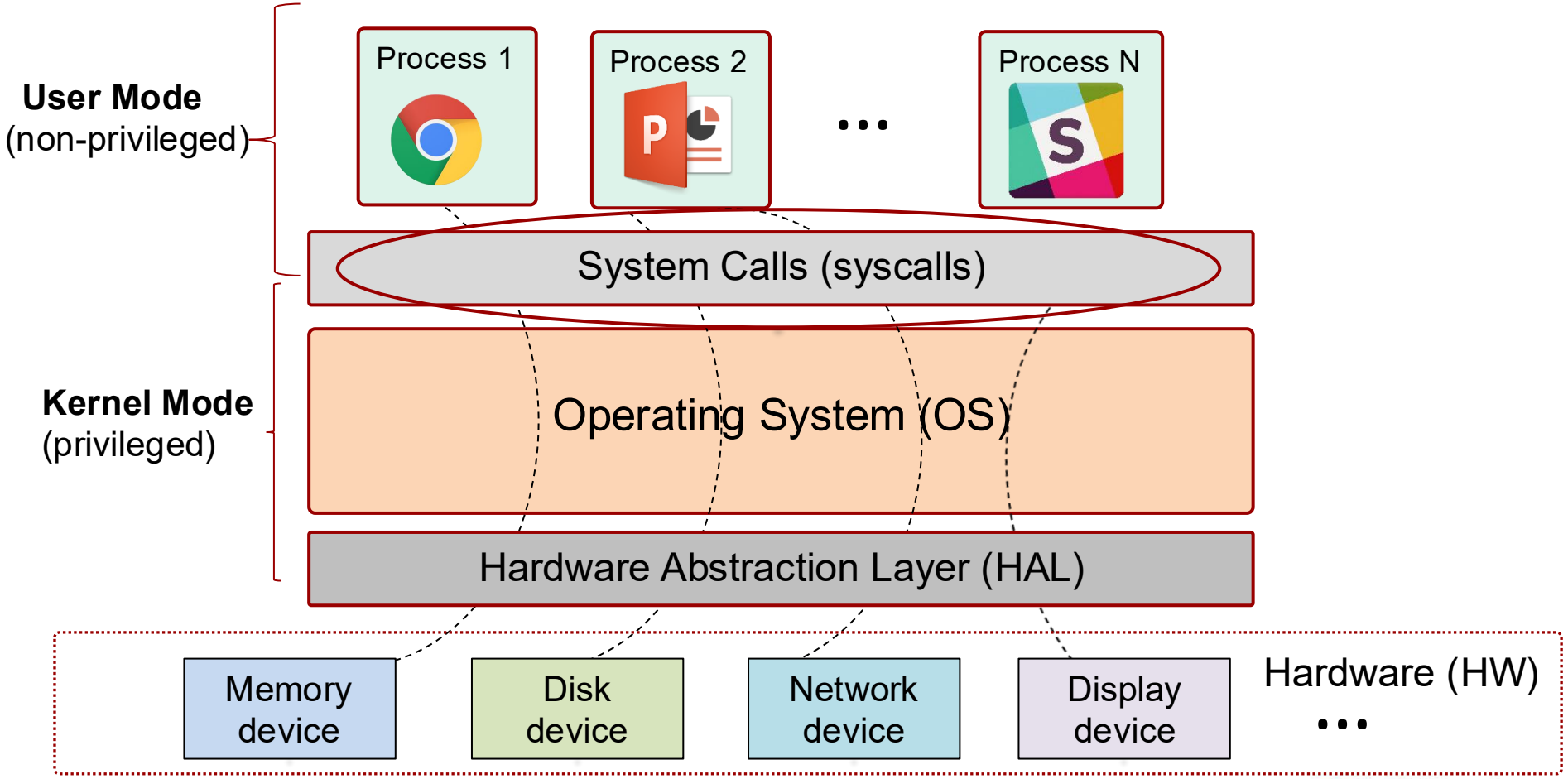
Lecture Outline

- Course information
- What is OS?
- Address space abstraction
- Process abstraction
- Syscall intro

© Tatyana Ryutov

Recall: What is Operating System (OS)?

- Software to manage computer resources for its users and applications



© Tatyana Ryutov

System Calls

- System Call (syscall) is an interface between a user app and a service provided by the OS
  - OS API from the user program's point of view, accessible via a library of code, e.g., **`libc`**
- Most common APIs:
  - Win32 API for Windows
  - POSIX API for POSIX-based systems
  - Java API for the Java virtual machine (JVM)
- Syscalls can be grouped into six major categories:
  1. Process control
  2. Device Management
  3. File Management
  4. Information Maintenance
  5. Communications
  6. Protection

© Tatyana Ryutov

Examples of Windows and Linux System Calls

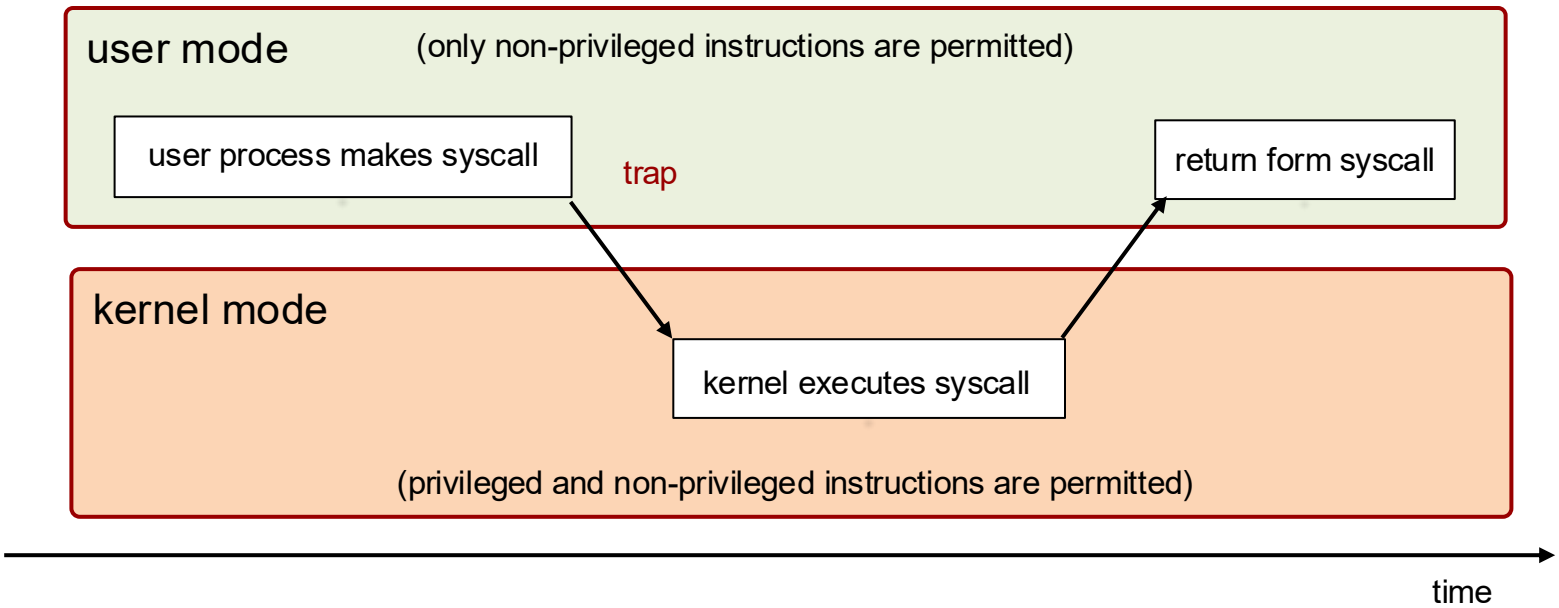
Types of System Calls	Windows	Linux
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
Device Management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
File Management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

USC Viterbi School of Engineering

© Tatyana Ryutov

Syscall Processing: Overview

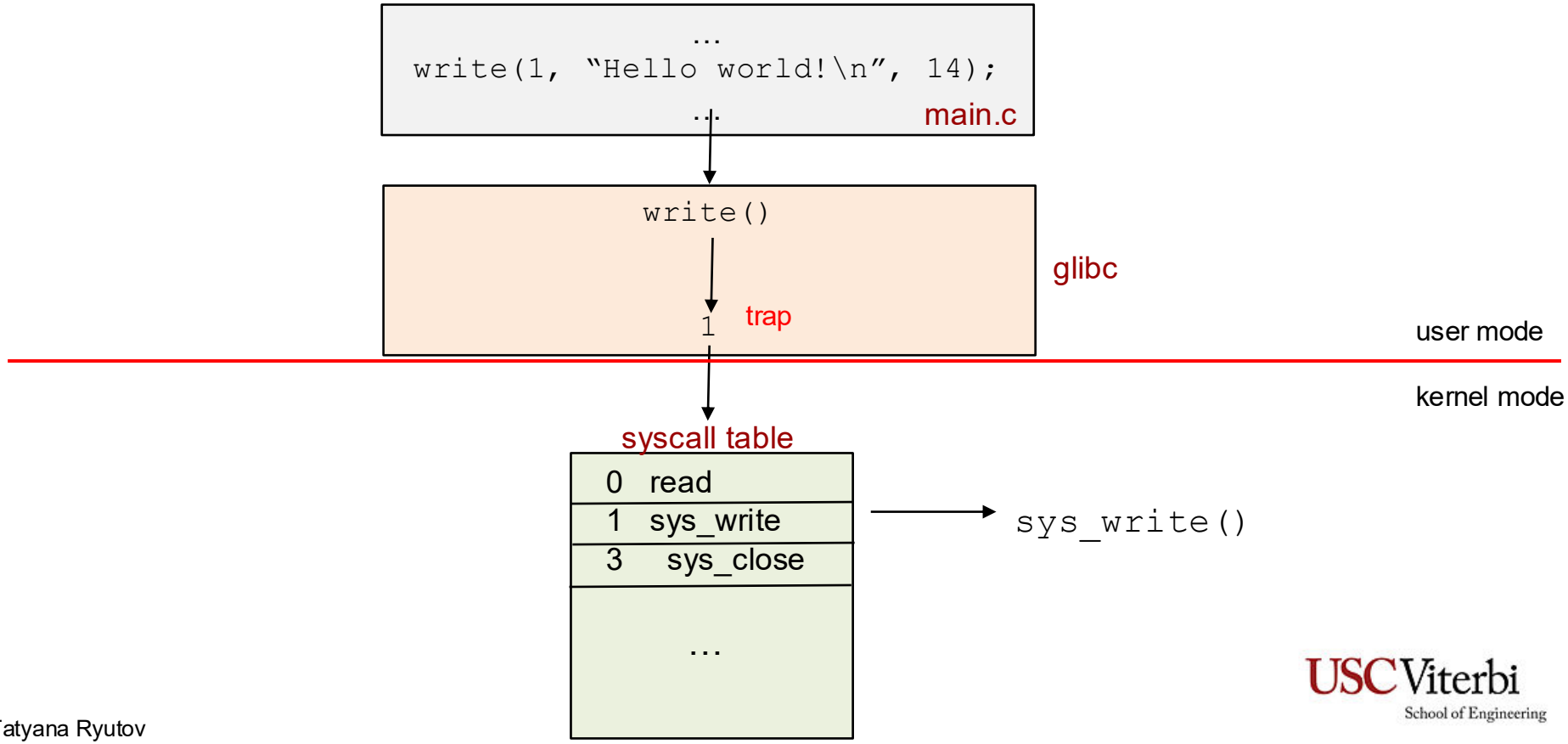
- System call generates a trap (a.k.a. supervisor call)
  - In x86 **`int`** instruction traps into the kernel on a syscall
  - In xv6 user-level program, **`int 0x40`** indicates a syscall
- The executing process is suspended
  - Control is transferred to the kernel that implements each syscall
- Process continues when syscall completes



© Tatyana Ryutov

How does OS know which code to call?

- A number is associated with each syscall
  - Syscall interface maintains a table indexed according to these numbers
  - Converts symbolic names of services to these numbers
- Example: **`glibc`** provides wrappers for syscalls that generate the actual syscalls



© Tatyana Ryutov

USC Viterbi School of Engineering

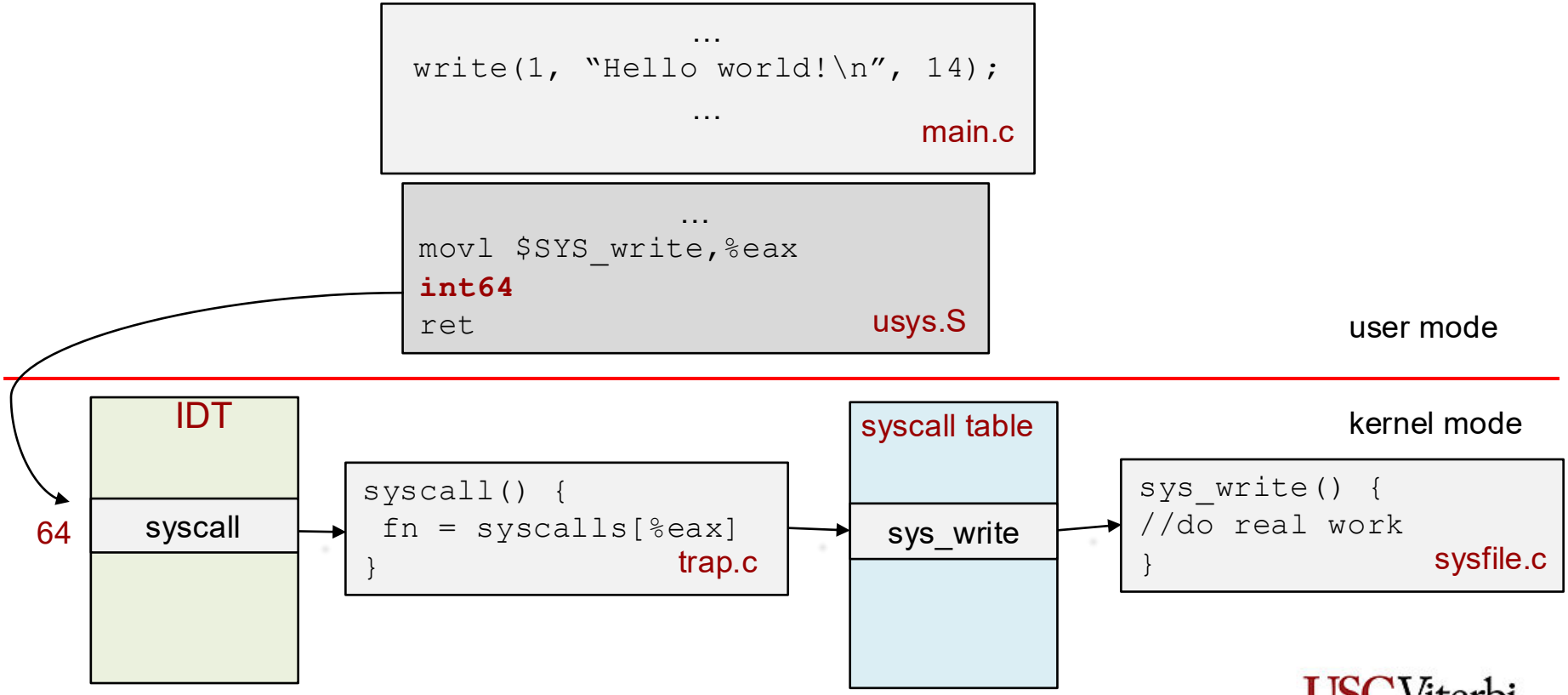
xv6 syscalls

System call	Description
int fork()	Create a process, return child's PID.
int exit(int status)	Terminate the current process; status reported to wait(). No return.
int wait(int *status)	Wait for a child to exit; exit status in *status; returns child PID.
int kill(int pid)	Terminate process PID. Returns 0, or -1 for error.
int getpid()	Return the current process's PID.
int sleep(int n)	Pause for n clock ticks.
int exec(char *file, char *argv[])	Load a file and execute it with arguments; only returns if error.
char *sbrk(int n)	Grow process's memory by n bytes. Returns start of new memory.
int open(char *file, int flags)	Open a file; flags indicate read/write; returns an fd (file descriptor).
int write(int fd, char *buf, int n)	Write n bytes from buf to file descriptor fd; returns n.
int read(int fd, char *buf, int n)	Read n bytes into buf; returns number read; or 0 if end of file.
int close(int fd)	Release open file fd.
int dup(int fd)	Return a new file descriptor referring to the same file as fd.
int pipe(int p[])	Create a pipe, put read/write file descriptors in p[0] and p[1].
int chdir(char *dir)	Change the current directory.
int mkdir(char *dir)	Create a new directory.
int mknod(char *file, int, int)	Create a device file.
int fstat(int fd, struct stat *st)	Place info about an open file into *st.
int stat(char *file, struct stat *st)	Place info about a named file into *st.
int link(char *file1, char *file2)	Create another name (file2) for the file file1.
int unlink(char *file)	Remove a file.

© Tatyana Ryutov

xv6: Syscall Processing (simplified)

- xv6 implements its syscalls similar to Linux, by having the software interrupt number **0x40** (decimal **64**) handled as a syscall
- Based on a specific syscall number stored in %eax register, the corresponding syscall handler is invoked



xv6: Syscall Registration

syscall numbers

```c
#define SYS_fork    1
#define SYS_exit    2
#define SYS_wait    3
#define SYS_pipe    4
#define SYS_read    5
#define SYS_kill    6
#define SYS_exec    7
#define SYS_fstat   8
#define SYS_chdir   9
#define SYS_dup    10
#define SYS_getpid 11
#define SYS_sbrk   12
#define SYS_sleep  13
#define SYS_uptime 14
#define SYS_open   15
#define SYS_write  16
#define SYS_mknod  17
#define SYS_unlink 18
#define SYS_link   19
#define SYS_mkdir  20
#define SYS_close  21
```

syscall.h

syscall handlers

```c
static int (*syscalls[])(void) = {
[SYS_fork]    sys_fork,
[SYS_exit]    sys_exit,
[SYS_wait]    sys_wait,
[SYS_pipe]    sys_pipe,
[SYS_read]    sys_read,
[SYS_kill]    sys_kill,
[SYS_exec]    sys_exec,
[SYS_fstat]   sys_fstat,
[SYS_chdir]   sys_chdir,
[SYS_dup]     sys_dup,
[SYS_getpid]  sys_getpid,
[SYS_sbrk]    sys_sbrk,
[SYS_sleep]   sys_sleep,
[SYS_uptime]  sys_uptime,
[SYS_open]    sys_open,
[SYS_write]   sys_write,
[SYS_mknod]   sys_mknod,
[SYS_unlink]  sys_unlink,
[SYS_link]    sys_link,
[SYS_mkdir]   sys_mkdir,
[SYS_close]   sys_close,
};
```

syscall.c

What should you do to add a new syscall?

© Tatyana Ryutov

xv6: Syscall Handling

```c
void
syscall(void)
{
  int num;

  // Get the syscall number from eax
  num = proc->tf->eax;

  // Check that the syscall number is greater than 0, inside the
  // syscalls and that a function pointer exists for that syscal
  if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    // Set eax to the return value of the syscall.
    proc->tf->eax = syscalls[num]();
  } else {
    // Else print an error and return -1 to the user program.
    cprintf("%d %s: unknown sys call %d\n",
            proc->pid, proc->name, num);
    proc->tf->eax = -1;
  }
}
```

syscall.c

© Tatyana Ryutov

USC Viterbi School of Engineering

xv6: Passing Syscall Parameters

- Arguments pushed onto user stack
- xv6 has its own built-in functions for fetching the arguments into a kernel function (see **`syscall.c`**)
  - E.g., to pass in an integer, **`argint()`** function is called

© Tatyana Ryutov